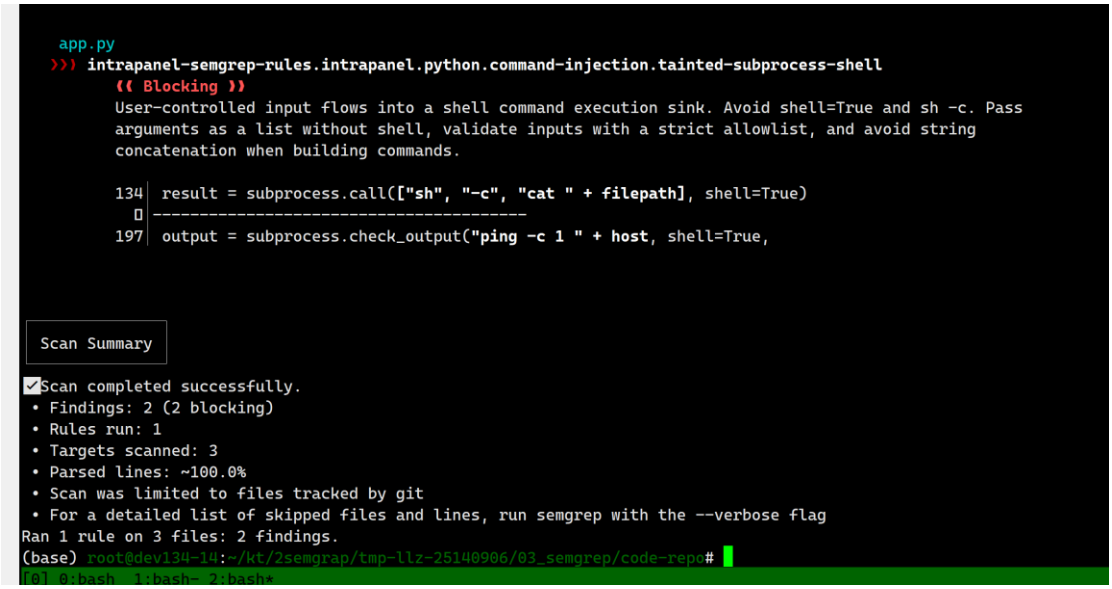
标题

作者

一、输出截图



二、形成规则

```
rules:
  - id: intrapanel.python.command-injection.tainted-subprocess-shell
    languages: [python]
    severity: ERROR
    mode: taint
    message: >-
      User-controlled input flows into a shell command execution sink.
      Avoid shell=True and sh -c. Pass arguments as a list without shell,
      validate inputs with a strict allowlist, and avoid string concatenation
      when building commands.
    metadata:
      category: security
      confidence: HIGH
      cwe: "CWE-78: Improper Neutralization of Special Elements used in an OS Command"
      owasp: "A03:2021 Injection"
      technology: [python, flask, subprocess]
      source: "Flask request input"
      sink: "subprocess/os shell execution"
      attack_flow: "HTTP parameter -> string/path concatenation -> shell interpreter"
    pattern-sources:
      # Flask request sources: query string, form body, combined values,
```

```
    # JSON body, headers, cookies, uploaded filename.
    - pattern: request.args.get(...)
    - pattern: request.args[...]
    - pattern: request.form.get(...)
    - pattern: request.form[...]
    - pattern: request.values.get(...)
    - pattern: request.values[...]
    - pattern: request.get_json(...)
    - pattern: request.json
    - pattern: request.headers.get(...)
    - pattern: request.cookies.get(...)
    - pattern: $FILE.filename

  pattern-sinks:
    # subprocess with shell=True: command string is interpreted by /bin/sh.
    - patterns:
      - pattern-either:
        - pattern: subprocess.run($CMD, shell=True, ...)
        - pattern: subprocess.call($CMD, shell=True, ...)
        - pattern: subprocess.check_call($CMD, shell=True, ...)
        - pattern: subprocess.check_output($CMD, shell=True, ...)
        - pattern: subprocess.Popen($CMD, shell=True, ...)
      - focus-metavariable: $CMD

    # Explicit shell invocation through sh -c / bash -c.
    # Dangerous even when shell=True is not present.
    - patterns:
      - pattern-either:
        - pattern: subprocess.run(["sh", "-c", $CMD], ...)
        - pattern: subprocess.call(["sh", "-c", $CMD], ...)
        - pattern: subprocess.check_call(["sh", "-c", $CMD], ...)
        - pattern: subprocess.check_output(["sh", "-c", $CMD], ...)
        - pattern: subprocess.Popen(["sh", "-c", $CMD], ...)
        - pattern: subprocess.run(["bash", "-c", $CMD], ...)
        - pattern: subprocess.call(["bash", "-c", $CMD], ...)
        - pattern: subprocess.check_call(["bash", "-c", $CMD], ...)
        - pattern: subprocess.check_output(["bash", "-c", $CMD], ...)
        - pattern: subprocess.Popen(["bash", "-c", $CMD], ...)
      - focus-metavariable: $CMD

    # Imported subprocess functions, e.g.:
    # from subprocess import check_output
    - patterns:
      - pattern-either:
        - pattern: run($CMD, shell=True, ...)
        - pattern: call($CMD, shell=True, ...)
        - pattern: check_call($CMD, shell=True, ...)
        - pattern: check_output($CMD, shell=True, ...)
        - pattern: Popen($CMD, shell=True, ...)
```

```
        - focus-metavariable: $CMD

    # Other direct shell execution APIs.
    - pattern: os.system(...)
    - pattern: os.popen(...)
    - pattern: commands.getoutput(...)
    - pattern: commands.getstatusoutput(...)
```

三、过程：命令注入漏洞说明

在本项目中，app.py 中存在命令注入风险，主要涉及系统诊断和文件下载两个功能点。该类漏洞的本质是：应用程序将来自 HTTP 请求的用户可控输入直接拼接到系统命令字符串中，并通过 subprocess 等接口交由操作系统 Shell 解释执行。当命令执行接口启用了 shell=True，或显式调用 sh -c、bash -c 等 Shell 解释器时，用户输入中的特殊字符可能被 Shell 解析为命令分隔符、命令替换、管道或重定向语法，从而导致攻击者执行非预期系统命令。

1、 漏洞成因

1) diag_ping 中的命令注入

```
@app.route("/diag/ping", methods=["GET", "POST"])
def diag_ping():
    if "user_id" not in session:
        return redirect(url_for("login"))
    output = None
    if request.method == "POST":
        host = request.form.get("host", "")
        # B602: subprocess with shell=True and unsanitized user input - command injection
        output = subprocess.check_output("ping -c 1 " + host, shell=True,
                                         stderr=subprocess.STDOUT).decode()
    return render_template("diag.html", output=output)
```

在 diag_ping 功能中，应用从请求参数中读取用户提交的主机地址，例如：

host = request.form.get("host", "")

该变量属于用户可控输入，即漏洞分析中的“源”（source）。随后程序将该输入直接拼接进系统命令：

"ping -c 1 " + host

并通过如下方式执行：

subprocess.check_output("ping -c 1 " + host, shell=True, ...)

其中，subprocess.check_output(..., shell=True) 是命令执行“汇”（sink）。由于启用了 shell=True，Python 不会仅把 host 当作 ping 命令

的普通参数处理，而是会将完整字符串交由 Shell 解释执行。因此，如果攻击者在 host 参数中注入 Shell 元字符，便可能改变原始命令语义，执行额外系统命令。该风险在上传的漏洞分析报告中也被列为 Critical 级命令注入问题。

2) files_download 中的命令注入

```
@app.route("/files/download")
def files_download():
    if "user_id" not in session:
        return redirect(url_for("login"))
    filename = request.args.get("file", "")
    # Path traversal + B602: shell=True with unsanitized user input
    filepath = os.path.join("uploads", filename)
    result = subprocess.call(["sh", "-c", "cat " + filepath], shell=True)
    return f"Exit code: {result}"
```

在 files_download 功能中，应用从查询参数中读取文件名：

filename = request.args.get("file", "")

该参数同样属于用户可控输入。程序随后将其拼接为文件路径：

filepath = os.path.join("uploads", filename)

再进一步拼接到 Shell 命令中：

"cat " + filepath

最后通过如下方式执行：

subprocess.call(["sh", "-c", "cat " + filepath], shell=True)

这里存在双重风险。第一，用户输入经过 os.path.join 后并未消除命令注入风险，因为路径字符串仍然会被拼接进 Shell 命令；第二，代码显式调用了 sh -c，使得拼接后的字符串被 Shell 解释。即使外层传递的是列表形式，只要内部使用了 sh -c，第三个参数仍然会被 Shell 当作命令脚本解析。因此，该路径同样构成“用户输入 → 字符串拼接 → Shell 执行”的危险污点流。

2、 攻击过程

该漏洞的攻击过程可以概括为以下几个阶段。

首先，攻击者通过 Web 接口提交恶意输入。例如，在诊断接口中，攻击者将原本应为主机地址的参数替换为包含 Shell 语法的字符串。应用程序未对该输入

进行严格校验，也未使用安全的参数化命令调用方式，而是将该输入直接拼接到系统命令中。

其次，拼接后的命令字符串被传入 subprocess.check_output、subprocess.call 等命令执行函数，并启用 shell=True 或通过 sh -c 调用 Shell。此时，Shell 会对命令字符串进行完整解析，攻击者输入中的命令分隔符、管道、命令替换符等会被解释为 Shell 控制语法。

最后，攻击者可以在应用进程权限范围内执行任意系统命令。根据运行权限和部署环境不同，可能造成敏感文件读取、系统信息泄露、服务破坏、横向移动、反向连接或进一步提权等后果。

3、 源、传播路径与汇

从静态分析角度看，该漏洞可以建模为污点传播问题。

类型	代码位置	示例	安全含义
Source	HTTP 请求参数	request.form.get("host") 、request.args.get("file")	攻击者可控输入
Propagation	字符串拼接/路径拼接	"ping -c 1 " + host 、os.path.join("uploads", filename)	用户输入被传播到命令字符串
Sink	Shell 命令执行	subprocess.check_output(..., shell=True)、subprocess.call(["sh", "-c", ...])	命令被 Shell 解释执行

因此，该类漏洞的核心判定条件不是“是否调用了 subprocess”，而是同时满足以下条件：

1. 命令内容包含用户可控输入;
2. 命令通过字符串拼接、格式化或其他方式构造;
3. 命令执行时经过 Shell 解释;
4. 缺少严格的白名单校验、参数化调用或安全转义机制。

4、 影响分析

命令注入属于高危甚至严重级别漏洞。一旦被成功利用，攻击者可能获得服务器上的命令执行能力。其影响范围通常包括：

1. **敏感信息泄露**: 攻击者可能读取系统文件、配置文件、环境变量、应用密钥或数据库连接信息。
2. **系统破坏**: 攻击者可能删除文件、修改应用代码、破坏数据或影响服务可用性。
3. **权限扩展**: 如果应用以高权限运行，攻击者可进一步扩大影响范围。

4. **持久化与横向移动**：攻击者可能写入后门、建立外连通道，或利用服务器作为跳板攻击内网资源。
5. **供应链与业务风险**：如果该系统部署在内部管理环境中，命令执行能力可能进一步威胁内部系统、凭证和业务数据。

5、 修复建议

修复该类问题应以“避免 Shell 解释”为核心原则。

对于 ping 类命令，应使用参数列表形式传递命令和参数，并关闭 shell=True：

```
subprocess.check_output(["ping", "-c", "1", host], shell=False)
```

同时，应对 host 参数进行严格白名单校验，仅允许合法 IP 地址或域名。例如，可使用 ipaddress 模块校验 IP，或使用严格正则校验域名格式。

对于文件下载功能，不应通过 cat 命令读取文件，而应使用 Python 原生文件读取能力。同时，需要结合路径规范化和目录边界检查，防止路径遍历：

```
base_dir = os.path.abspath("uploads")
target_path = os.path.abspath(os.path.join(base_dir, filename))

if not os.path.commonpath([base_dir, target_path]) == base_dir:
    raise ValueError("Invalid file path")
```

此外，应避免以下危险写法：

```
subprocess.call("cat " + filepath, shell=True)
subprocess.call(["sh", "-c", "cat " + filepath])
subprocess.check_output("ping -c 1 " + host, shell=True)
```

推荐修复原则如下：

修复方向	说明
禁用 shell=True	默认使用 shell=False，避免 Shell 解释用户输入
使用参数列表	将命令和参数分离，例如 ["ping", "-c", "1", host]
白名单校验	对 IP、域名、文件名进行严格格式限制
避免命令替代业务逻辑	文件读取应使用 Python 文件 API，而不是调用 cat
最小权限运行	Web 服务进程不应以 root 或高权限账户运行
审计日志	记录异常参数和命令执行失败情况，便于溯源

6、 Semgrep 检测思路

针对该漏洞，可以使用 Semgrep 的 taint mode 建立检测规则，将 Flask 请求参数作为 source，将 subprocess、sh -c、bash -c、os.system 等命令执

行接口作为 sink。当用户输入能够传播到命令执行接口时，即判定为潜在命令注入风险。

该规则的检测重点应覆盖以下模式：

1. request.args.get(...)、request.form.get(...)、request.values.get(...) 等 HTTP 输入;
2. 字符串拼接、格式化、路径拼接等传播路径;
3. subprocess.*(..., shell=True);
4. subprocess.*(["sh", "-c", ...]) 或 subprocess.*(["bash", "-c", ...]);
5. os.system(...)、os.popen(...) 等直接 Shell 执行接口。

通过这种方式，可以较准确地识别“用户输入流入 Shell 命令执行”的高危路径，并降低仅凭关键词匹配造成的误报。